

Student Course Allocation System

Brief Architecture Document

1. Architecture Design

The Student Course Allocation System follows a three-tier architecture consisting of the Presentation Layer, Application Layer, and Database Layer.

- **Presentation Layer (Frontend):**
 - Developed using React.js.
 - Provides an interactive and responsive user interface for students and administrators.
 - Sends HTTP requests to the backend through REST APIs.
- **Application Layer (Backend):**
 - Developed using Node.js and Express.js.
 - Handles authentication, business logic, course allocation, AI requests, and database operations.
 - Uses JWT for secure user authentication.
 - Uses Socket.IO to provide real-time updates.
- **Database Layer:**
 - PostgreSQL is used as the relational database.
 - Stores user accounts, student information, course details, allocations, and application data.
 - SQL queries are executed through the backend to maintain data consistency.

Overall Flow

1. User logs into the application.
2. React sends API requests to the Express server.
3. Express validates the JWT token.
4. Business logic is executed.
5. PostgreSQL stores or retrieves data.
6. Response is returned to the frontend.
7. Socket.IO sends real-time updates whenever required.
8. AI requests are forwarded to the OpenAI API and the response is displayed to the user.

2. Database Design Decisions

PostgreSQL was selected because it provides:

- Strong ACID compliance.
- Excellent relational data management.
- High reliability.
- Efficient indexing and query performance.
- Easy scalability for future enhancements.

The database is normalized to reduce redundancy and improve maintainability.

The major entities include:

- Users
- Students
- Courses
- Course Allocations

Primary keys uniquely identify each record, while foreign keys maintain relationships between tables.

Indexes are used on frequently searched columns to improve query performance.

3. AI Integration Approach

The system integrates the OpenAI API to provide AI-assisted functionality.

The backend receives prompts from the frontend and securely forwards them to the OpenAI API.

The AI module can assist users by:

- Answering academic queries.
- Providing intelligent suggestions.
- Supporting course-related recommendations.
- Improving the overall user experience.

The AI service is isolated from the core application logic, making it easy to maintain or replace in the future.

4. Security Considerations

Several security measures have been implemented throughout the application.

Authentication

- JWT-based authentication.
- Protected API routes.

- Session validation.

Password Security

- Passwords are encrypted using bcrypt before being stored in PostgreSQL.

Environment Variables

Sensitive information such as:

- Database credentials
- JWT Secret
- OpenAI API Key

are stored in environment variables instead of the source code.

API Security

- Request validation.
- Proper HTTP status codes.
- Unauthorized access prevention.

Database Security

- Parameterized SQL queries to reduce SQL injection risks.
- Controlled database access through backend APIs only.

5. Challenges Faced and Solutions Implemented

Challenge 1

Integrating React, Node.js, and PostgreSQL together.

Solution

Designed a clear REST API structure and separated frontend and backend responsibilities.

Challenge 2

Connecting the AI service with the application.

Solution

Created a dedicated backend service to communicate with the OpenAI API, ensuring API keys remain secure.

Challenge 3

Managing database relationships.

Solution

Designed normalized PostgreSQL tables with primary and foreign key constraints to maintain data integrity.